

Lab & Activities Week 04 Retrieval-Augmented Generation (RAG)

Integrantes:

José Martínez-02220131016

Institución: Universidad de Santander

Pregrado: Ingeniería de Sistemas

Docente: Jorge Galvis

28 de Agosto 2026

Activity 1 · Warm-up Concept Quiz

Activity 1 · Warm-up Concept Quiz10 MININDIVIDUALFORMATIVE

1. Which two RAG stages run on every single user query (not just once at setup)?

- ☐ Ingestion and indexing
- ☒ Retrieval and generation
- ☐ Chunking and embedding

Correct answer: B. Ingestion/indexing happen once per document; retrieval/generation happen per query.

2. Why add overlap between chunks in fixed-size chunking?

- ☐ To reduce the chance of cutting a relevant idea across a chunk boundary
- ☐ To make the vector store smaller
- ☐ Overlap is required by ChromaDB's API

Correct answer: A. Overlap trades a little duplication for fewer split-idea failures.

3. Your assistant confidently answers a question your source documents never mention. What should you check first?

- ☐ Increase the temperature
- ☐ Switch from ChromaDB to FAISS
- ☒ Print what was actually retrieved and check the "answer only from context" instruction is in the prompt

Correct answer: C. Most "hallucination" bugs in student RAG systems are retrieval bugs.

4. What's the main practical advantage of ChromaDB over rolling your own FAISS index for a typical class project?

- ☐ ChromaDB is faster at every scale
- ☒ Built-in persistence and metadata filtering with a simple Python API
- ☐ FAISS cannot store text, only ChromaDB can

Correct answer: B. FAISS is a lower-level library; you'd build persistence/metadata yourself.

Check answers Score: 4 / 4

Activity 2 · Instructor-Led Live Coding

Follow along in week-04-rag.ipynb, sections 1-3: chunk a sample document, index it in both ChromaDB and FAISS, and run `answer_with_rag` live, including on a deliberately out-of-scope question, to see the "I don't have that information" behavior in action.

1. Chunking

```
1. Chunking

def chunk_text(text: str, chunk_size: int = 400, overlap: int = 60) -> list[str]:
    text = ".join(text.split()) # normalize whitespace
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunks.append(text[start:end])
        start = end - overlap
    return chunks

chunks = chunk_text(COURSE_DOCS, chunk_size=400, overlap=60)
print(f"({len(COURSE_DOCS)}) chars -> ({len(chunks)}) chunks")
for i, c in enumerate(chunks):
    print(f"({i}) {c[:70]}...")

1) ✓ 00s Python

...
2137 chars -> 7 chunks
[0] AI Agent Engineering is a 16-week elective course for Systems Engine...
[1] s 12-16) covers evaluation, observability, and deployment to productio...
[2] s chosen by the student, using the Gemini API and ChromaDB, delivered ...
[3] capabilities, delivered with a system diagram and a 15-minute live de...
[4] 5-minute demo video and a 20-minute technical presentation. All labs ...
[5] d coding agent that can scaffold and edit project files. Class time ea...
[6] the course GitHub repository, with Moodle used for supporting materia...
```

With `chunk_size=400` and `overlap=60`, a 2137-character document becomes 7 chunks. Each chunk starts 340 characters after the previous one ($400 - 60$), so the last 60 characters of one chunk always repeat as the first 60 of the next -- exactly the mechanic behind quiz question 2.

2. Indexing with ChromaDB and FAISS

```
2. Indexing with ChromaDB

import chromadb

chroma_client = chromadb.PersistentClient(path="./chroma_db_week4")
# start fresh each run of this notebook
try:
    chroma_client.delete_collection("course_docs")
except Exception:
    pass
collection = chroma_client.create_collection(name="course_docs")

collection.add(
    documents=chunks,
    embeddings=[embed(c) for c in chunks],
    ids=[f"chunk-{i}" for i in range(len(chunks))],
)
print(collection.count(), "chunks indexed")

1) ✓ 57s Python

7 chunks indexed

2a. The same vectors in FAISS (for comparison)

import faiss

vectors = np.array([embed(c) for c in chunks], dtype="float32")
faiss_index = faiss.IndexFlatL2(vectors.shape[1])
faiss_index.add(vectors)

query_vector = np.array([embed("How is Corte 1 graded?")], dtype="float32")
distances, indices = faiss_index.search(query_vector, k=3)
for dist, idx in zip(distances[0], indices[0]):
    print(f"({dist:.3f}) {chunks[idx][:80]}...")

1) ✓ 36s Python

0.684 s 12-16) covers evaluation, observability, and deployment to production. Corte 1...
0.716 capabilities, delivered with a system diagram and a 15-minute live demonstratio...
0.739 s chosen by the student, using the Gemini API and ChromaDB, delivered as a GitHub...
```

FAISS returns L2 distances (lower = closer), the opposite direction from the cosine similarity scores used in Week 3 (higher = closer) -- easy to misread if you're not paying attention to which metric a given library returns. ChromaDB's `collection.count()` confirms the same 7 chunks got indexed on that side too, and both retrieve the same top chunks for this query, since they're built from the exact same embeddings.

3. answer_with_rag

```
3. A complete answer_with_rag function

def answer_with_rag(question: str, n_results: int = 3, verbose: bool = False) -> str:
    results = collection.query(query_embeddings=embed(question), n_results=n_results)
    retrieved_chunks = results["documents"][0]
    if verbose:
        print("--- retrieved chunks ---")
        for c in retrieved_chunks:
            print("-", c[:80], "...")
    context = "\n\n---\n\n".join(retrieved_chunks)

    prompt = f'''Answer the question using ONLY the context below. If the answer isn't
    in the context, say you don't have that information - do not make anything up.

    Context:
    {context}

    Question: {question}
    Answer:'''

    response = client.models.generate_content(
        model=MODEL, contents=prompt,
        config=types.GenerateContentConfig(temperature=0.1),
    )
    return response.text

print(answer_with_rag("How is Corte 1 graded?", verbose=True))
print()
print(answer_with_rag("What programming language does the course use?")) # not in the docs -> should decline

[5] ✓ 57s Python

Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend to use AFC in Chat.send_message. Similarly, direct use of AFC in Models.generate_content is not recommended.

--- retrieved chunks ---
- s 12-16) covers evaluation, observability, and deployment to production. Corte 1 ...
- capabilities, delivered with a system diagram and a 15-minute live demonstratio ...
- s chosen by the student, using the Gemini API and ChromaDB, delivered as a GitHub ...
Based on the provided context, Corte 1 is worth 30% of the final grade, broken down as follows:
* **25%** for a practical project
* **5%** for in-class activities (such as quizzes, workshops, and labs)

I don't have that information.
```

The in-scope question gets a grounded, detailed answer built only from the retrieved chunks. The out-of-scope question ("What programming language does the course use?", never stated anywhere in COURSE_DOCS) gets a clean decline instead of a guess -- exactly the behavior quiz question 3 is about: this only works because the prompt explicitly says "if the answer isn't in the context, say you don't have that information", not because the model is naturally cautious.

Activity 4 · Individual Lab: Complete the Notebook + Apply to Your Project

Sections 4: ADK tool / LangChain preview

Note: cell as given has the same `asyncio.run(...)-inside-a-running-loop` bug already fixed in Weeks 2 and 3 -- rewrote `ask_adk_agent` as `async def`, used `await/async` for `on_create_session` and `run_async`, and called it with a top-level `await`.

4. Light preview: RAG as an ADK tool, and the LangChain equivalent

We're one step away from a real agent (Week 6 onward covers tool use and ReAct properly). For now, notice that "retrieval" is just a Python function — which is exactly what an ADK **tool** is.

```
from google.adk.agents import Agent
from google.adk.runners import InMemoryRunner

def retrieve_context(query: str) -> dict:
    """Retrieve the most relevant course-document chunks for a query."""
    results = collection.query(query_embeddings=[embed(query)], n_results=3)
    return {"chunks": results["documents"][0]}

rag_agent = Agent(
    model=MODEL,
    name="course_rag_agent",
    instruction=(
        "Use the retrieve_context tool to find relevant chunks before answering. "
        "Answer ONLY using information returned by the tool. If it's not there, say so."
    ),
    tools=[retrieve_context],
)

async def ask_adk_agent(agent, prompt, app_name="week4_app", user_id="student"):
    runner = InMemoryRunner(agent=agent, app_name=app_name)
    session = await runner.session_service.create_session(app_name=app_name, user_id=user_id)
    content = types.Content(role="user", parts=[types.Part.from_text(text=prompt)])
    final_text = None
    async for event in runner.run_async(user_id=user_id, session_id=session.id, new_message=content):
        if event.content and event.content.parts and event.content.parts[0].text:
            final_text = event.content.parts[0].text
    return final_text

print(await ask_adk_agent(rag_agent, "How is Corte 2 graded?"))
```

Python

✓ 65s

[C:\Users\José Martínez\Desktop\cosas de la U\ai_agentic\venv\Lib\site-packages\google\adk\models\llm_request.py:273: UserWarning: \[EXPERIMENTAL\] Feature FeatureName.JSON_SCHEMA_FOR_FUNC_DECL is declaration = tool.get_declaration\(\)](#)

Corte 2 is worth **30%** of the final grade and is broken down as follows:

- 25%** - Multi-Agent System Project: Students must build a multi-agent system that solves a real problem using Google ADK or LangGraph, integrating RAG capabilities. It is delivered with a sy
- 5%** - In-Class Activities: Quizzes, workshops, and labs.

Same grounded-answer pattern as Activity 2, just reached through a tool call instead of a direct retrieval + prompt: the ADK agent decides on its own to call `retrieve_context` before answering, which is the same underlying mechanic as a RAG pipeline, framed as an agent tool -- exactly what the notebook's own note says is one step away from a real agent.

Own project documents (calculus tutoring assistant)

My Corte 1 project (from the Week 1 proposal) is a calculus tutoring assistant. Since gathering the real lecture notes/textbook chapters from Week 3 is still pending, I just ask to the AI to give me a 3 short original placeholder notes (limits.txt, derivatives.txt, integrals.txt, in data/) covering the actual scope from my Week 2 system prompt, just so the pipeline could be exercised on real, domain-relevant text instead of the generic course-description sample. These are explicitly a stand-in and will be replaced once the real materials are gathered.

5. Exercises

```
# Exercise 1 - Run this on YOUR OWN Corte 1 project documents
# Project: calculus tutoring assistant; data/ has 3 short placeholder notes (limits.txt,
# derivatives.txt, integrals.txt) standing in for the real lecture notes/textbook chapters
# (gathering the real materials is still pending from the Week 3 checkpoint).

import glob

DATA_DIR = "data"
doc_paths = sorted(glob.glob(os.path.join(DATA_DIR, "*.txt")))
print("Files found:", [os.path.basename(p) for p in doc_paths])

calc_chunks = []
calc_sources = []
for path in doc_paths:
    with open(path, "r", encoding="utf-8") as f:
        text = f.read()
        file_chunks = chunk_text(text, chunk_size=400, overlap=60)
        calc_chunks.extend(file_chunks)
        calc_sources.extend([os.path.basename(path)] * len(file_chunks))

print(f"({len(calc_chunks)}) chunks total from ({len(doc_paths)}) files")

calc_chroma_client = chromadb.PersistentClient(path="./chroma_db_week4_calc")
try:
    calc_chroma_client.delete_collection("calculus_docs")
except Exception:
    pass
calc_collection = calc_chroma_client.create_collection(name="calculus_docs")

calc_collection.add(
    documents=calc_chunks,
    embeddings=[embed(c) for c in calc_chunks],
    ids=[f"chunk-{i}" for i in range(len(calc_chunks))],
    metadatas=[{"source": s} for s in calc_sources],
)
print(calc_collection.count(), "calculus chunks indexed")

def answer_calc_rag(question, n_results=3, where=None):
    kwargs = {"query_embeddings": [embed(question)], "n_results": n_results}
    if where:
        kwargs["where"] = where
    results = calc_collection.query(**kwargs)
    context = "\n\n---\n\n".join(results["documents"][0])
    prompt = f'''Answer the question using ONLY the context below. If the answer isn't
in the context, say you don't have that information - do not make anything up.

Context:
{context}

Question: {question}
Answer:'''
    response = client.models.generate_content(
        model=MODEL, contents=prompt,
        config=types.GenerateContentConfig(temperature=0.1),
    )
    return response.text

questions = [
    "What is the chain rule and when do I use it?",
    "Why do we always add + C when finding an indefinite integral?",
    "What's the difference between a limit existing and a function being continuous at that point?",
    "How does the Fundamental Theorem of Calculus connect derivatives and integrals?",
    "When should I use integration by parts instead of substitution?",
]
for q in questions:
    print(f"Q: {q}")
    print(f"A: {answer_calc_rag(q)}")
```

```
def answer_calc_rag(question, n_results=3, where=None):
    kwargs = {"query_embeddings": [embed(question)], "n_results": n_results}
    if where:
        kwargs["where"] = where
    results = calc_collection.query(**kwargs)
    context = "\n\n---\n\n".join(results["documents"][0])
    prompt = f'''Answer the question using ONLY the context below. If the answer isn't
in the context, say you don't have that information - do not make anything up.

Context:
{context}

Question: {question}
Answer:'''
    response = client.models.generate_content(
        model=MODEL, contents=prompt,
        config=types.GenerateContentConfig(temperature=0.1),
    )
    return response.text

questions = [
    "What is the chain rule and when do I use it?",
    "Why do we always add + C when finding an indefinite integral?",
    "What's the difference between a limit existing and a function being continuous at that point?",
    "How does the Fundamental Theorem of Calculus connect derivatives and integrals?",
    "When should I use integration by parts instead of substitution?",
]
for q in questions:
    print(f"Q: {q}")
    print(f"A: {answer_calc_rag(q)}")
```

✓ 242s

Files found: ['derivatives.txt', 'integrals.txt', 'limits.txt']
13 chunks total from 3 files
13 calculus chunks indexed

Q: What is the chain rule and when do I use it?
A: Based on the provided context:

* **What it is:** If $f(x) = g(h(x))$, then $f'(x) = g'(h(x)) * h'(x)$.
* **When to use it:** It is used when differentiating composite functions (functions with an "outer" and "inner" function).

Q: Why do we always add + C when finding an indefinite integral?
A: Based on the provided context, we always include "+ C" because two antiderivatives of the same function can only differ by a constant (representing the entire family of antiderivatives of the

Q: What's the difference between a limit existing and a function being continuous at that point?

LLM Responses the answers like this:

Q: What is the chain rule and when do I use it?

A: Based on the provided context, the chain rule is the rule where if $f(x) = g(h(x))$, then $f'(x) = g'(h(x)) * h'(x)$. You use it for composite functions.

Q: Why do we always add + C when finding an indefinite integral?

A: We always add "+ C" (an arbitrary constant of integration) because two antiderivatives of the same function can only differ by a constant, and the indefinite integral represents the entire family of antiderivatives of that function.

Q: What's the difference between a limit existing and a function being continuous at that point?

A: A limit existing at a point c only requires that both one-sided limits exist and are equal. A function being continuous at c is stricter: $f(c)$ must be defined, the limit must exist, and the limit must equal $f(c)$. The limit existing is only one of the three conditions for continuity.

Q: How does the Fundamental Theorem of Calculus connect derivatives and integrals?

A: If F is an antiderivative of f , the definite integral of $f(x) dx$ from a to b equals $F(b) - F(a)$ -- this makes it possible to evaluate definite integrals directly from an antiderivative instead of computing an area numerically.

Q: When should I use integration by parts instead of substitution?

A: Integration by parts is useful for products of functions (like $x \cdot e^x$) and reverses the product rule, while substitution is used when the integrand contains a function and its derivative (reversing the chain rule). (The context didn't give further criteria beyond these two descriptions.)

All 5 answers are grounded correctly in the 3 placeholder documents, with no fabricated content -- a good sign the pipeline itself is sound and ready to point at the real course materials once they're gathered.

Exercise 2 -- metadata filtering

Re-indexed the same 13 chunks with `metadatas=[{"source": "...}]`, then compared an unfiltered query against one restricted with `where={"source": "integrals.txt"}`.

```
# Exercise 2 - Metadata filtering
# calc_collection was already indexed with metadata={"source": "..."} in Exercise 1.

print("--- query with no filter (searches all 3 documents) ---")
results = calc_collection.query(query_embeddings=[embed("rate of change")], n_results=3)
for doc, meta in zip(results["documents"][0], results["metadatas"][0]):
    print(f"[meta['source']] {doc[:70]}...")

print("\n--- same query, filtered to source=integrals.txt only ---")
results = calc_collection.query(
    query_embeddings=[embed("rate of change")],
    n_results=3,
    where={"source": "integrals.txt"},
)
for doc, meta in zip(results["documents"][0], results["metadatas"][0]):
    print(f"[meta['source']] {doc[:70]}...")

print("\nFiltered answer using only Integrals.txt:")
print(answer_calc_rag("What does this document say about rate of change?", where={"source": "integrals.txt"}))

[18] ✓ 60s Python
```

```
--- query with no filter (searches all 3 documents) ---
[derivatives.txt] Derivatives The derivative of a function f at a point x measures the i...
[integrals.txt] te the volume of a solid of revolution, the total displacement of an o...
[integrals.txt] onstant. The definite integral of f(x) from a to b, written as the int...

--- same query, filtered to source=Integrals.txt only ---
[integrals.txt] te the volume of a solid of revolution, the total displacement of an o...
[integrals.txt] onstant. The definite integral of f(x) from a to b, written as the int...
[integrals.txt] ls: it states that if F is an antiderivative of f, then the definite i...

Filtered answer using only Integrals.txt:
I don't have that information.
```

The unfiltered query for "rate of change" pulls from both derivatives.txt and integrals.txt, since rate of change is central to both. With the where filter locked to integrals.txt, only that document's chunks come back -- and asking directly "what does this document say about rate of change" against that filtered context correctly returns "I don't have that information", because integrals.txt talks about accumulation and area, not instantaneous rate of change (that concept lives in derivatives.txt, which the filter excluded). This is actually a good outcome: the filter is doing exactly what it should, and the system declines instead of stretching an adjacent but different document to answer something it doesn't actually say.